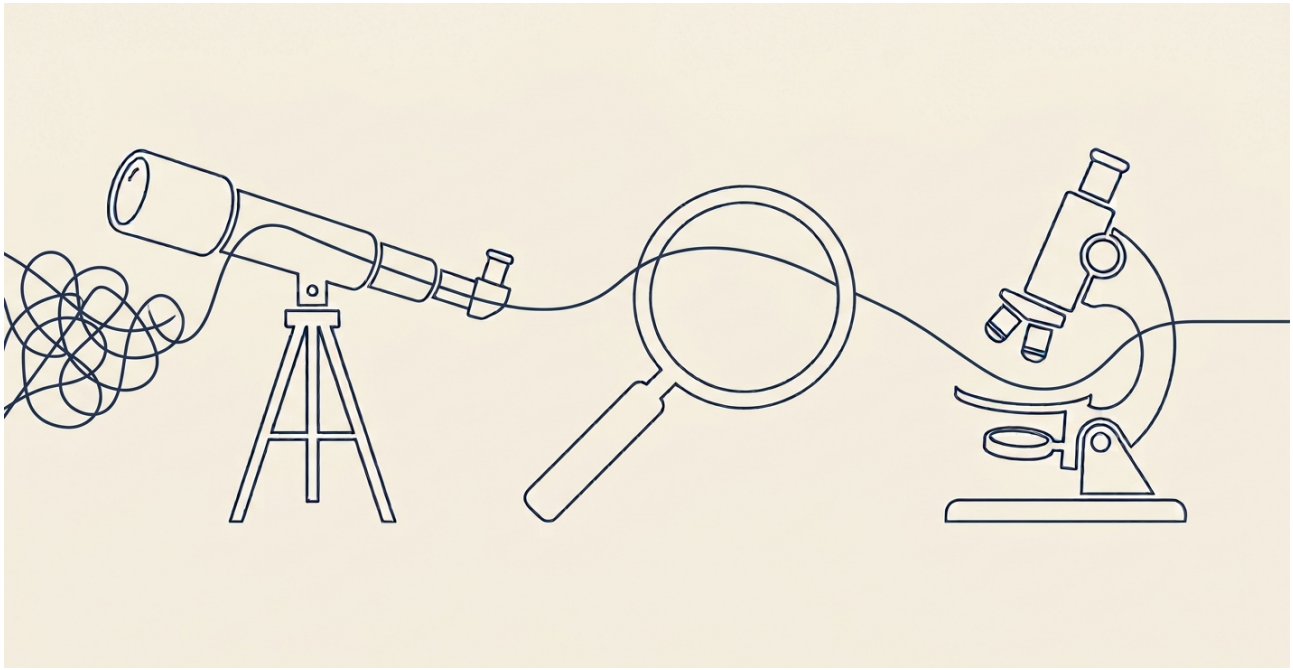


Council-Ontology

July 19, 2026

Alex Karp, CEO of Palantir, has publicly emphasised the value of their Ontology as a semantic layer over other approaches. According to Palantir leadership, it's why they "won". Because ontology allows you to do things alternatives can't: it can create verifiable, auditable chains of reasoning. Decision pathways taken by AI agents on ontological data are auditable because ontology gives every step a stable, typed referent (a fact slotted into a defined category, rather than free text). AI reasoning upon ontological data increases accountability between the system's claims, and the system which verifies them - because claims can be checked against an independent source of truth. Ontology establishes the structure for that truth to be accurately represented - and I've seen the opposite.

About six months ago, when working on a self-led project at work to package up disorganised logs and trace errors across disparately connected systems, I witnessed an LLM model in action performing an agentic loop that planned, reasoned across steps, and pulled data through tool use, such as search engines, collecting from disparate online sources in an ad hoc fashion. The tool I made, while useful, relied on AI-generated insights, based on AI-generated analysis, based on a mix of AI-collected data and real system log data. The insights were useful tip-offs as to why the system failed - but not proofs, and with bedrock assumptions (like which subsets of data to aggregate) taken for granted by a behind-the-scenes agentic workflow, I had no real way to probe and validate the chains of reasoning behind those insights. The result was an opaque system which helped reduce manual investigation times, not a fully fledged auditing tool.

To me, the 'magic' of LLMs is their ability to turn large amounts of unstructured information into structured information. An LLM can act as a bridge between non-ontological data and an ontology. The amount of unstructured data in the world is seemingly endless. That it is now feasible to contain and organise that data - to make it legible to vast systems of computation, and therefore to artificial intelligence - is a

breakthrough we're still yet to comprehend the effects of. One might reasonably guess that intelligence agencies are rapidly expanding their ability to digitally perceive the analogue world. Hence, retail access to frontier AI puts a one-man intelligence agency within reach, at least based over open source information. And what are the open sources of information? Public data, such as local government council meeting minutes, are downloadable from the internet, and contain the inner workings of councils across Australia. I took it upon myself to develop an ontology of Councils - bridging the unstructured records of meeting minutes with a stable, formalised database.

The project is open source: https://github.com/j-dabrowski/council-ontology

Anchored Claims

The key to verifiable chains of reasoning is making claims that can be verified. "Motion 5 was defeated" is a claim which can point to a PDF, a page, a line number, a quote. The evidence behind the claim is traceable to ground truth. The more reasoning a claim integrates, the less anchored its output is to the source evidence - to verify such a claim requires more deductive reasoning, as opposed to a simple comparison of fact-to-source check. A singular claim with a scope too large, such as "The system failed because of subsystem A's schedule collided with subsystem B" makes big claims without sourcing timing logs, or evidence of a collision stack trace, and does no work to rule out other possible explanations. So to construct a pipeline of unstructured meeting minutes to structured council information in a database, I constrained the task of an LLM as one module in the pipeline such that it can only produce anchored claims. Every 'fact' presented by an LLM must also point to a source quote - which is then verified programmatically. Verified data can be built upon to make further inferences. This is what enables higher levels of abstraction and research - the fact that each inference in a causal chain of inferences, which produces a large scale inference, can be rebuilt, and checked against its source. From "Motion 5 was defeated", all the way to "Motion 5 was set up to fail". The building of an inference has two halves - building facts you can trust, and then reasoning over them - and this piece is about the first, the substrate the reasoning runs on.

The Open World Problem

A known philosophical problem in intelligence analysis is the open-world assumption, where you don't know what you don't know - versus the closed-world assumption, where unknowns are considered false. In a closed world, the information we do know is exhaustive, complete, and if something did not exist within our corpus, it would be considered to not exist at all. In practice, this would mean if a financial interest is not declared, the assumption would be that there definitively wasn't one, whereas an open-world assumption caveats that with "not to our knowledge". We do live in an open world. Inferences can only be made on information we have, leaving open the possibility that conflicting inferences may be formed on information we don't have. The best inference over a currently known corpus could be defeated by evidence not yet collected. This means an inference generated by the system is auditable, but not 'final'. This is 'defeasibility', it is a natural consequence of the open world.

Since we can't exit the open world, the search needs a rule for when to stop - an implicit or explicit benchmark of what counts as having looked hard enough. In Council-Ontology, the scraping phase gathers the corpus that everything downstream will reason over, and that gathering is governed by such a benchmark. A naive example of a benchmark being

twelve sets of council minutes per year, or one per month. The benchmark makes the absence of a PDF from the corpus detectable, and it scopes every later claim by defining the quality of the corpus those claims were drawn from. This is how "we don't know what we don't know" becomes "there is a gap, and we can aim to close it", and declaring the gap is what gives us any honest basis for rating confidence in an insight. The benchmark is itself an educated guess about an open world, and can still be wrong. Processing the City of Cambridge, the twelve-per-year assumption was disproved by the extracted data itself, which revealed that Cambridge holds no January meetings, and some months have no ordinary meeting at all.

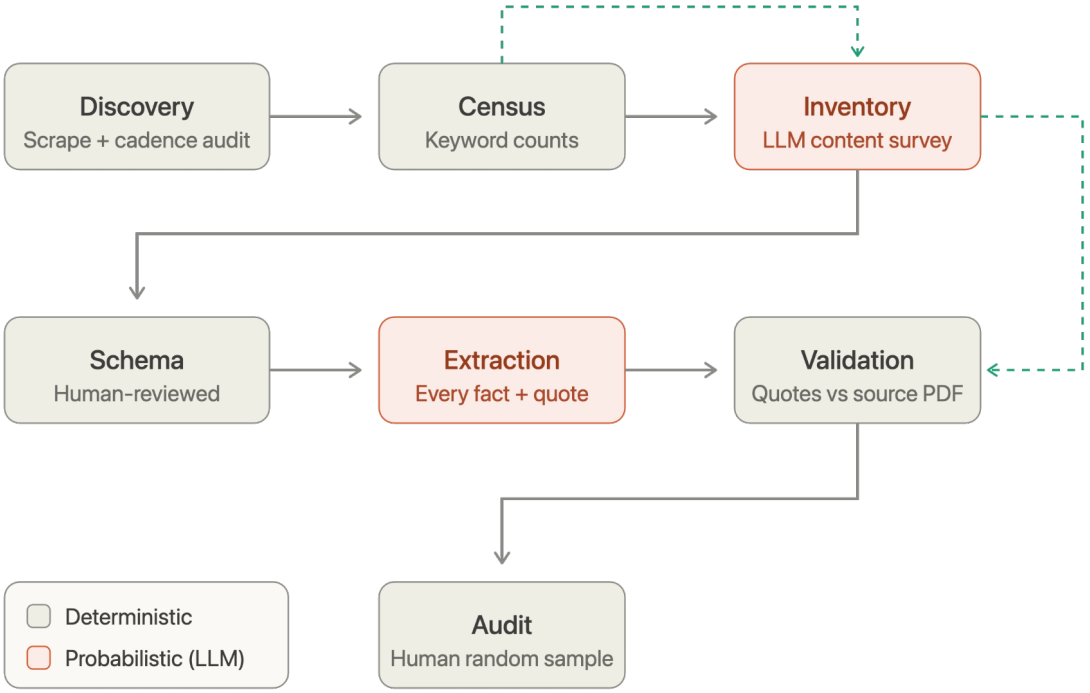
Every insight generated by this system is limited to the corpus we found. Within that boundary, there are three types of information gap, with differing levels of detectability. Most detectable is content inside documents we did gather but failed to extract, which we built metrics to catch. Less detectable are documents we did not gather, but had reason to expect in the corpus - an expectation set by external research and discovery, not one made implicit by the corpus itself. Least detectable are governance decisions and events which were never entered into the meeting minutes, and stay off the record. These leave no trace in any of the available information, so there can be no basis for forming an expectation that they exist. The first two are more explicit failures that present an engineering problem. The third, an epistemic problem - a limit, rather than something we can solve.

To solve those first two limitations, the pipeline was driven by a simple mechanism: for any gap that may pass silently, run it through a formalised metric to detect whether it happened or not. Wherever we have information that can tell us what expectation to build, we build a metric to expect that. This principle shaped the pipeline into a sequence of stages, each paired with metrics that check it.

The Pipeline

The stages divide into two types. Deterministic stages - code, regex, human judgment - produce the same output given the same input and carry no inherent risk of invention. Probabilistic stages are LLM calls: capable of hallucination, silent omission, and output that varies run to run. Only the probabilistic stages carry quality metrics, because metrics are only warranted where the output is uncertain. Several deterministic stages are also expectation discovery stages: their output becomes the benchmark a later probabilistic stage is held against - always built from information the model never saw, so the check remains independent of the thing it checks.

Two timelines run through this pipeline. Before any probabilistic stage is trusted to run at scale, it goes through a threshold-gated improvement loop: run the stage on a sample, measure its metrics, edit the prompt, re-run, compare scores, repeat until performance converges on an acceptable threshold, then freeze. The metrics that drive these loops are the same ones used to verify the stage in production - they function as exit criteria during development and as quality checks at runtime. The stage descriptions below tell the second timeline: the pipeline as it runs in production, where probabilistic stages operate within deterministically-set expectations and are verified against them. By the time a stage appears in the sequence below, its improvement loop has already run and its prompt has already converged.



The two LLM stages are checked against expectations produced by deterministic stages that ran before them. Dashed arrows mark checks the model never saw, so it cannot fit its output to them.

Runtime Stages

Discovery (scraper) - *deterministic, expectation discovery*: A crawler pulls every meeting PDF the council's site exposes; a cadence audit (the scraper audit, in the codebase) flags any year that falls short of the council's real meeting rhythm. A missing document is invisible from inside the corpus, so the expectation must come from outside it: Cambridge is required to hold at least one ordinary meeting per month, January excepted, and the audit holds each scraped year against that floor. It currently flags 2022 and 2023, where four to five consecutive months have no meetings found - traced to a council website migration in mid-2022 that neither the new site nor web archives cover. The gap is recorded, so any conclusion drawn from 2022 or 2023 carries a known caveat: the underlying year is incomplete.

Census - *deterministic, expectation discovery*: A deterministic keyword scan counts what each PDF appears to contain. These counts can be used as expectations for what we should find in an LLM-based extraction: X-many instances of the word ‘Councillor’, Y-many instances of the word ‘vote’, etc. Counts are never included in the model's prompt, to avoid ‘reward-hacking’ by the model. If the model knew the numbers it would be checked against, it could shape its output to satisfy them rather than to reflect the document - an extraction that passes the check without being correct. Withholding them keeps the check independent of the output it verifies: the model can only hit the expected counts by actually extracting what is there.

Inventory - *probabilistic, expectation discovery*: other_content_rate measures the catch-all 'other' category, where the LLM places content the current schema has no category for. It indicates that the schema lacks an ontological framework for something the corpus actually contains. It is an unknown unknown in the ontology. We decrease the other_content_rate by iterating on the prompt to better ask for what's in the documents that wasn't previously asked for. Once "other" is minimal, there is a measured basis for confidence that the extraction prompt is not blind to a major class of content - the system's ignorance was quantified, then closed.

Schema - *deterministic*: The extraction schema is written from what the inventory found, instead of being guessed beforehand. Writing a schema before reading the corpus amounts to guessing at what the documents contain, whereas writing it after is closer to transcription. The catch-all 'other' category from the inventory functions as an alert or warning - anything that lands in it is content the current schema did not anticipate, surfaced rather than silently dropped, and each schema revision aims to empty it.

The schema update uses Claude Code to read the inventory's output and write the resulting changes into the codebase. There is an LLM in that step. But the question of whether something is deterministic isn't really about whether an LLM was involved - it's about whether the output is reviewed and ratified before it has downstream effects. In the schema update, a human sees every change before it's committed. The LLM's variance - and there is some - is absorbed at that review step and never propagates. Compare that to extraction, where 580 documents run without per-output human review and whatever the model decided becomes the data. The difference isn't the presence of an LLM; it's whether a human stands between the model's output and the thing that output affects. Human-in-the-loop at the decision point makes a process effectively deterministic. Human-in-the-loop only at the audit stage - after the fact, on a sample - does not, because it lets changes flow through the system unchecked until the end, by which point it's too late to catch a bad extraction before it becomes a fact the rest of the system trusts.

Extraction - *probabilistic*: Probabilistic component, batch-run, every fact forced to arrive holding a source quote. This is the stage with the greatest capacity to hallucinate, which is why it is the most heavily constrained. Trust is not placed in the model's assertion but instead in the source quote it is required to produce alongside every fact, which is then verified programmatically. This creates a single probabilistic component (extraction via LLM) bounded on both sides by deterministic code.

Validation - *deterministic, verification*: Every quote the model emitted is matched back against the source PDF. The metrics split across two failure modes: a model that says something false, and a model that says nothing where something should have been extracted. Precision metrics catch the first, whereas the second requires proxies, because an omission leaves nothing to measure directly against. The first five metrics below gate the extraction convergence loop; Entity Density and Schema Completeness are corpus-level integrity checks and run at validation time but do not gate the loop.

- **Quote Completeness** - fraction of entities with ≥1 source quote. Catches the failure to evidence at all - an entity with no source quotes may be hallucinated or may be real but unsubstantiated; either way it has not met the standard of evidence.

- **Paraphrase Rate** - fraction of quotes not matchable in normalised source text after exhaustive matching. The direct fabrication check: if the model's quote cannot be found in the source PDF, the claim was never there.
- **Inventory Agreement** - extracted entity counts vs the L1 inventory counts, flagged if ratio <0.4 or >2.5. The L1 inventory is generated cheaply and independently before extraction runs; the expensive extraction is then held against it. This is the same "independent expectation" pattern as the census, applied at the entity level rather than the keyword level - a check the model can't fit itself to because it never saw the inventory.
- **Keyword Gap Rate** - occurrences of key terms (e.g. MOVED, CARRIED, DEVELOPMENT APPLICATION, DECLARATION OF INTEREST, DEPUTATION, PETITION) in the source text not spanned by any matched quote. When a quote is verified against the source, it is pinned to a span - the start-and-end character range it occupies in the document text. The union of those spans marks everything in the document that some extracted entity accounts for. Keyword Gap Rate counts key terms falling outside every span: places where the source says MOVED or DECLARATION OF INTEREST but no extracted entity's evidence touches that part of the text. These indicate keywords the model should have extracted something about, but didn’t. It is a rough heuristic, as a keyword can appear in procedural context without a corresponding entity, but it is the closest available signal for something real being there and nothing being said about it.
- **Coverage Ratio** - the fraction of the document's text that falls inside a matched quote span. This sits low (~13%) across the corpus, and low is correct: most of a minutes document is procedural boilerplate - attendance lists, standing orders, formatting - that no entity should be extracted from. The substantive content (motions, votes, declarations) occupies a small fraction of the text, so a well-behaved extraction quotes that fraction and ignores the rest. The metric is not a target to maximise, but it does have a floor: a document falling far below the corpus norm (under ~2-3%) is flagged for review or failure, as it signals the model may have skipped real content.
- **Entity Density** - motions per 10k chars; flags large Ordinary meetings with suspiciously few motions. A corpus-level sanity check for quiet failure: a meeting from which almost nothing was extracted asserts nothing false, so paraphrase rate misses it entirely. Density is the document-level proxy for that form of silent failure.
- **Schema Completeness** - Ordinary meetings must have ≥1 motion; all motions must have a non-null outcome. A structural integrity check: these are things the minutes format guarantees, so their absence in the extracted data is unambiguous signal that something went wrong.

Audit - *deterministic, ground-truth verification*: A human marks each entity against the source on a random sample. Every automated recall check is a proxy, and a proxy that is never ground-truthed is a number taken on faith. The audit establishes whether the proxies track reality - and because it samples randomly rather than drawing only from documents the validation metrics flagged for review, it is the one stage capable of surfacing a gap no test was written to expect. A sample built only from flagged documents would inherit the blind spots of the metrics that did the flagging.

Each stage of the pipeline turns a predictable absence into a metric, and the third and least detectable gap (events that never entered any record) trips no metrics, because no expectation can be built for something that left no trace.

Convergence Loops

Each stage below reaches production the same way: not when its output starts to feel good, but when a measured threshold is crossed - a passing cadence audit, an other_content_rate at or below 20%, the five core validation metrics inside their targets. The threshold is the exit criterion: once crossed, the stage's improvement loop ends and the pipeline proceeds to the next stage. One distinction to keep in view: these loops are not all the same kind of work. The two probabilistic stages converge a prompt - the LLM's behaviour is tuned until its output meets the metric. The Discovery loop has no prompt to tune; it is code being debugged against a target, a gap being chased until it is either closed or documented as unrecoverable. Convergence and debugging both exit on a measured threshold, which is why they belong together here, but only one of them involves tuning a model's behaviour.

Discovery loop: If the cadence audit flags a year below the expected meeting count, the scraper code is inspected: selectors checked against the live site, URL patterns re-probed, Wayback CDX queried for archived copies. Confirmed gaps that can't be recovered from online sources are documented and the council contacted directly. Unlike the LLM loops, there is no prompt to tune - only code to fix or a gap to declare unrecoverable.

Inventory loop: After each run, the typology analysis script computes other_content_rate and generates a prompt describing what the current schema missed. That prompt is fed to Claude Code, which edits inventory_prompt.txt; the prompt version is bumped, invalidating the cache and forcing a fresh LLM call on the next run. The loop repeats until other_content_rate falls to ≤20%. Only then does the inventory prompt freeze and the schema get written. The exit criterion is what makes it rigorous: the prompt is not judged adequate by feel; the loop exits when the measured catch-all rate empties.

Extraction loop: Before the full corpus batch runs, the extraction prompt is tested against an 18-document stratified sample. Extract the sample, run validate-sample, read the paraphrase report to identify failure patterns, edit system_prompt.txt, re-extract with --force, re-validate, compare scores against the previous run. The loop runs until the five core validation metrics converge within their targets. Only then is the prompt trusted at scale. Running the loop on a small sample first means the expensive batch doesn't run until the prompt has already proven itself on the representative sample.

Results

The pipeline has now worked through 580 documents from the City of Cambridge - minutes, agendas, and addenda spanning meetings from April 1995 to June 2026, with a known and documented gap across parts of 2022-2023, where a council website migration left several months unrecoverable from any online source. It extracted 14,013 motions and 16,249 recorded votes involving 400 councillors. The volume matters less than the sourcing: of the roughly 45,000 entities pulled out of those documents - motions, planning applications, public questions, tenders, budget items, and the rest - 98 percent carry at least one verbatim quote back to the source PDF, pinned to a character offset in the original text (counting every entity once across the whole corpus - a different figure from the per-document averages below). The remainder are flagged as exactly that: extracted without a quote, visible as a gap rather than passing as fact. Nothing in the dataset is asked to be trusted on the model's word alone.

Volume tells us nothing about whether an extraction can be trusted, so every document is scored against its own text - how much of what was extracted is backed by a locatable quote, how much was paraphrased rather than lifted, and whether the entity counts agree with an independent inventory pass taken before extraction ran. Those scores roll up into a per-document verdict: PASS, REVIEW where a metric warrants human attention, or FAIL where thresholds are breached badly enough that the document's extraction cannot be trusted as-is. The numbers split cleanly along the tuning boundary. The extraction prompt was iteratively refined through its convergence loop against the 87 documents from 2024 onward, then frozen and run unmodified across the remaining 493 documents spanning 1995-2023. On the tuned set, quote completeness averages 98 percent, paraphrase sits near 6 percent, and no document is marked FAIL. On the 493 older documents - three decades of formats the prompt was never tuned for - quote completeness averages 81 percent, and 141 are marked FAIL. The 81 percent is the more meaningful number, because it measures performance on documents the prompt was never directly tuned against.

Because every fact is typed and sourced, the corpus stops being a pile of PDFs and becomes something that can be queried and reasoned over: which motions carried, which lost, who moved what, and how council's decisions compared to what officers recommended going in. That last comparison already runs - matching agenda recommendations to minutes outcomes across 203 paired motions, council followed the officer's recommendation 97 percent of the time. What that number means - whether it shows elected members simply ratifying what the administration puts in front of them, and whether that reading survives analysis of the 3 percent that diverged - is the reasoning layer built on top of this substrate, and a separate question from the one this piece set out to answer.

The tool I built at work reasoned over data it had gathered and analysed itself. When an insight came out the other end, there was no independent source to check it against - only more AI output underneath it. This project is the reverse. The reasoning layer is currently being built, and it will reason over a corpus where every fact is typed, carries a source quote, and can be checked against the original PDF. The difficult part was never going to be the reasoning. It was building a set of facts solid enough that anything reasoned on top of them can be traced back and verified, rather than trusted because a model said so. Three decades of council minutes, previously unstructured and unread at scale, are now a queryable database where every entry traces back to its source, and where the gaps are marked rather than hidden. What gets built on top of that substrate is the subject of the next article - but the question only makes sense once the facts underneath it can be trusted.

Code, metrics, and the full pipeline: https://github.com/j-dabrowski/council-ontology

Josef Dabrowski is a Perth-based software engineer building verifiable LLM extraction systems.